

Instruments Playground SDK

Python API 说明文档

目录

Instruments Playground SDK Python API 说明文档	1
1. 设备控制	2
2. 示波器 (Oscilloscope)	4
2.1 示波器配置:	4
2.2 触发 (Trigger)	4
2.3 采样率 (Sample Rate) 与缓冲区大小 (Buffer Size)	6
2.4 运行与信号采集	7
3. 电源 (Power Supply)	10
4. 任意波形发生器 (Arbitrary Waveform Generator, AWG)	11
4.1 节点 (Node)	11
4.2 扫描模式 (Sweep mode)	14
5. 数字静态 IO (Static IO)	16
5.1 如何访问数字 IO 端口	16
6. 逻辑分析仪 (Logic Analyzer)	17
7. 码型发生器 (Pattern Generator)	20
8. 协议分析仪 (Protocol Analyzer)	22
9. 数字万用表 (Digital Multimeter, DMM)	26

1. 设备控制

DeviceCount(preamble: bool = False) -> None

查询当前通过 USB 端口连接的雨珠设备数量，并将结果存储在 self.devicecount 中。

参数

preamble: bool

若为 True，则额外以文本形式（打印到终端或日志）输出查询结果。默认为 False。

示例

```
>>> from pyRD import RD
>>> rd = RD()
b'RDSDKVERSION3.0'
>>> rd.DeviceCount(True)
c_long(2)
>>> print(rd.devicecount)
2
```

DeviceEnumLists() -> None

查询所有已连接的雨珠设备，并将其描述信息（Description）和序列号（Serial Number）以结构化数组形式存储于 self.devicelist 中。

数据结构

self.devicelist: List[List[bytes, bytes]]

- 每个子列表表示一个设备，格式为 [描述信息, 序列号]。
- 描述信息和序列号均为 bytes 类型，需使用 decode() 函数可以将其转换为 str 类型。

示例

```
>>> from pyRD import RD
>>> rd = RD()
b'RDSDKVERSION3.0'
>>> rd.DeviceEnumLists()
>>> print(rd.devicelist)
[[b'USB <-> Serial Converter', b'YZH00895'], [b'USB <-> Serial
Converter', b'YZJ00770']]
```

DeviceOpen(index: int) -> int

打开指定索引的雨珠设备，并返回设备状态。若操作成功，会额外以文本形式输出 "open device success"。

参数

index: int

目标设备在 self.devicelist 列表中的索引值（从 0 开始）。若仅连接了一台雨珠设备，通常为 0。

示例

```
>>> from pyRD import RD
>>> rd = RD()
b'RDSDKVERSION3.0'
>>> status = rd.DeviceOpen(0)
open device success
>>> print(status)
0
```

`DeviceClose()` -> `int`

解除对当前已打开的雨珠设备的占用，使其可被其他程序或进程（如 IP）重新访问。

示例

```
>>> from pyRD import RD
>>> rd = RD()
b'RDSDKVERSION3.0'
>>> status = rd.DeviceOpen(0)
open device success
>>> rd.DeviceClose()
0
```

2. 示波器 (Oscilloscope)

2.1 示波器配置：

`AnalogInCHEnable(ch: int, enable: bool) -> int`

设置示波器某通道的模拟信号输入功能的开关状态。

参数

ch: int

要选择的通道。

ch	示波器通道
0	通道 1/通道 A
1	通道 2/通道 B
2	通道 3/通道 C
3	通道 4/通道 D

enable: bool

要设置的开关状态，True 为开启，False 为关闭。

`AnalogInCHRangeSet(ch: int, Vrange: float) -> int`

设置示波器某通道的电压量程。

参数

ch: int

要选择的通道。

ch	示波器通道
0	通道 1/通道 A
1	通道 2/通道 B
2	通道 3/通道 C
3	通道 4/通道 D

Vrange: float

要选择的量程，单位为 V。除雨珠 2 外，雨珠设备的示波器有 5V 和 25V 两个量程。雨珠 2 的示波器只有 1 个 10V 量程，不需要进行此设置。

2.2 触发 (Trigger)

触发通过同步电压和时间数据，将波形固定在屏幕上某一电压/时间点，便于分析。触发模式 (Trigger Modes) 决定示波器如何开始扫描信号，目前有 3 种模式：

a. 无触发源模式 (None Mode)

示波器不会等待触发条件满足，而是持续扫描信号。

配置方法：`AnalogInTriggerSourceSet(RDTRIGSRCNone)`

b. 自动模式 (Auto Mode)

示波器会等待触发，但如果一段时间内没有达到触发条件，示波器也会扫描信号。

配置方法：

- 设置有效触发源
- `AnalogInTriggerAutoTimeoutSet(>0)`

c. 普通模式 (Normal Mode)

仅当信号达到触发条件时才开始扫描信号，否则屏幕空白或保持上一次捕获的波形。

配置方法：

- 设置有效触发源
- `AnalogInTriggerAutoTimeoutSet(0)`

`AnalogInTriggerSourceSet(trigsrc = RDTRIGSRCNone) -> int`

设置示波器的触发源，可以选择各输入通道作为触发源或外部触发源，也可以选择无触发源。选择无触发源时触发模式为无触发源模式。

参数**trigsrc**

要选择的触发源，默认值为无触发源。

trigsrc	触发源
RDTRIGSRCNone	无
RDTRIGSRCDetectorAnalogInCH1	示波器通道 1
RDTRIGSRCDetectorAnalogInCH2	示波器通道 2
RDTRIGSRCDetectorAnalogInCH3	示波器通道 3
RDTRIGSRCDetectorAnalogInCH4	示波器通道 4
RDTRIGSRCEXternal1	外部触发源 1
RDTRIGSRCEXternal2	外部触发源 2

`AnalogInTriggerAutoTimeoutSet(timeout: int = 0) -> int`

当 `timeout > 0` 时：设置示波器等待触发的超时时间，超时后示波器将强制触发，并开始扫描输入信号，此时触发模式为自动模式；

当 `timeout == 0` 时：示波器会永远等待触发条件达成，此时触发模式为普通模式。

参数**timeout: int**

要设置的超时时间，单位为秒 (s)，默认值为不会超时。

`AnalogInTriggerTypeSet(trigtype = RDTRIGTYPEEdge) -> int`

设置示波器的触发方式。当前仅支持边沿触发 (Edge Triggering)，所以也可以不进行本设置。未来可能会增加脉冲触发 (Pulse Triggering) 等其他触发方式。

参数**trigtype:**

要选择的触发方式，默认值为边沿触发。

trigtype	触发方式
RDTRIGTYPEEdge	边沿触发

```
AnalogInTriggerConditionSet(slope = RDTriggerSlopeEdge) -> int
```

设置示波器的触发方向，可以选择上升沿、下降沿或双边沿触发。

参数

slope:

要选择的触发方向，默认值为双边沿触发。

slope	触发方向
RDTriggerSlopeRise	上升沿
RDTriggerSlopeFall	下降沿
RDTriggerSlopeEdge	双边沿

```
AnalogInTriggerLevelSet(voltsLevel: float = 0, voltRange: int = 5) -> int
```

若选择的触发源为内部触发源，可以设置示波器的触发电压。

参数

voltsLevel: float

要设置的触发电压，根据触发源的量程取值范围为 -5V ~ +5V 或 -25V ~ +25V，默认值为 0V；

voltRange: int

选择的触发源通道的量程（5V 或 25V），默认值为 5V。

2.3 采样率（Sample Rate）与缓冲区大小（Buffer Size）

a. 定义：

采样率（sample rate）是示波器每秒对输入信号进行模数转换（ADC）的次数，单位为 Sa/s（Samples per second）或 Hz。采样率受硬件（主要是 ADC 的性能）限制。雨珠设备的最大采样率有 2 种版本：40MHz 和 125MHz，可以通过设备的序列号来分辨。通常情况下，序列号以 YZA, YZJ 与 YZH 开头的设备为 40MHz，以 YZD 与 YZF 开头的为 125MHz。实际使用时，采样率可以在 1Hz~最大采样率（40MHz/125MHz）之间调节。

缓冲区大小（buffer size）指示波器在采集信号时用于存储采样数据的存储容量，以采样点（samples）为单位表示。通常情况下，雨珠设备的缓冲区最大为 4096 个采样点，除雨珠 2 外，缓冲区大小可以调节；雨珠 2 的缓冲区固定为 4096 个采样点。

b. 影响：

采样率越高，相邻采样点的时间间隔越小，对快速变化的信号，如高频信号或边沿陡峭的信号的还原能力越强。此外，根据奈奎斯特定律（Nyquist theorem），要想无失真地还原信号，采样率需要 $\geq$ 信号最高频率的 2 倍（实际工程中常需要 $\geq 2.5 \sim 5$ 倍），比如一个 40MHz

的雨珠设备的示波器可以捕获到频率最高为 20MHz 的正弦波信号。

但同时采样率也不是越高越好，过高的采样率会降低示波器可捕获的信号时长，即时间窗口（time window）。时间窗口由采样率和缓冲区大小共同决定，可表达为：

$$\text{时间窗口} = \text{缓冲区大小} / \text{采样率}$$

更低的采样率和更大的缓冲区大小（更多的采样点）都可以延长时间窗口，以便测量在时域上跨度更大的信号。但是使用更大的缓冲区也会增加每一次采集信号的时间（死区时间）。综上，需要根据待测信号的频率和其他特征，选择合适的采样率和缓冲区大小。

AnalogInFrequencySet(fre: int) -> int

设置示波器的采样率（sample rate）。

参数：

fre: 采样率的范围是 1Hz~40MHz 或 1Hz~125MHz，上限取决于使用的硬件型号。采样率的取值需要遵守如下原则： $fre = \frac{freMax}{div}$ ，其中 fre 为采样率且为整数，freMax 为采样率上限，div 为分频参数且为正整数。

AnalogInBufferSizeSet(bufferize: int = 2048) -> int

设置示波器的缓冲区大小（buffer size）。

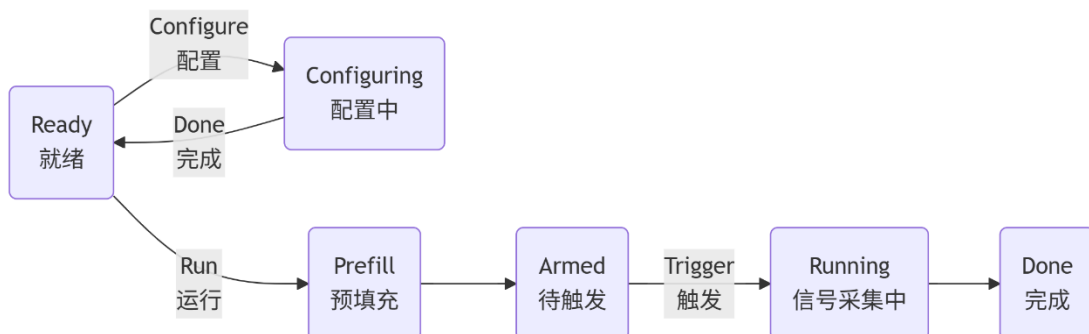
参数：

bufferize: 可选值为 32, 64, 128, 256, 512, 1024, 2048, 4096。雨珠 2 固定为 4096 个采样点，不需要进行此设置。

2.4 运行与信号采集

2.4.1 常规模式

常规模式下示波器的运行状态：



```
AnalogInRun(run: bool = True) -> int
```

命令示波器开始运行，直到达到 Done 状态；或命令示波器立即停止运行。

参数：

run: True 为开始运行，False 为停止运行。

```
AnalogInStatus() -> int
```

读取示波器的当前状态。

返回值：

返回以 int 类型表示的设备状态，每个数字对应示波器的一种或多种状态。RDconstant 文件中定义了每种状态的助记常量，比如 RDStateReady = 0。对应关系参见下表：

int	RDconstant	示波器状态
0	RDStateReady	准备完成，等待操作
1	RDStateArmed	等待触发条件满足
2	RDStateDone	信号采集完成
3	RDStateRunning	信号采集中
3	RDStateTriggered	触发条件满足，触发完成
4	RDStateConfig	设备配置中
5	RDStatePrefill	预填充触发前所需的样本缓冲
7	RDStateWait	

```
AnalogInRead(size: int = 2048, ch: int = 0) -> int
```

读取示波器相应通道的缓冲区所存储的数据，读取后会清空缓冲区，结果保存在 rd.aidatach1 ~rd.aidatach4 中。

参数：

size: 设置用于保存读取到的数据的数组大小，通常应与缓冲区大小相等。如果不相等，则可能会截断读取到的数据或在其末端补 0。

ch: 示波器通道的索引编号，对应关系参见下表：

ch	数据保存位置	示波器通道
0	rd.aidatach1	通道 1/通道 A
1	rd.aidatach2	通道 2/通道 B
2	rd.aidatach3	通道 3/通道 C
3	rd.aidatach4	通道 4/通道 D

2.4.2 滚动移位模式 (Shift Mode)

滚动移位模式 (shift mode) 是一种连续采集模式，在这种模式下，当缓冲区存满时，最旧的片段会被覆盖，新片段从时间轴末端加入，形成“滚动移位”效果。这种模式专为长时间监测缓慢变化的信号而设计。


```
AnalogInShiftModeRun(enable: bool = True) -> int
```

命令示波器开始或停止以滚动位移模式运行。此模式下，示波器开始运行后会持续不断地采集信号，直到接收到停止运行的命令。此模式最高支持 4KHz 采样率。

参数：

run: True 为开始运行，False 为停止运行。

```
AnalogInShiftRead(ch: int = 0) -> int
```

读取示波器在滚动位移模式下某通道的缓冲区所存储的数据，读取后会清空缓冲区。对于通道 1，采样结果保存在 `rd.aidatach1` 中，采样点数保存在 `rd.aibacksizech1.value` 中。读取后需要进一步处理得到的数据，即取采样结果的前采样点数项，在 python 中可以使用 `list(rd.aidatach1)[:rd.aibacksizech1.value]`；

此外，需要注意在滚动位移模式下，缓冲区大小为 512 个采样点，当采集到的数据超出缓冲区大小时，最旧的采样点会被丢弃，因此读取数据的时间间隔应小于

$$\text{临界时间间隔} = \text{缓冲区大小} / \text{采样率}$$

否则会出现部分数据丢失的现象。

参数：

ch: 示波器通道的索引编号，对应关系参见下表：

ch	数据保存位置	更新的数据长度保存位置	示波器通道
0	<code>rd.aidatach1</code>	<code>rd.aibacksizech1.value</code>	通道 1/通道 A
1	<code>rd.aidatach2</code>	<code>rd.aibacksizech2.value</code>	通道 2/通道 B
2	<code>rd.aidatach3</code>	<code>rd.aibacksizech3.value</code>	通道 3/通道 C
3	<code>rd.aidatach4</code>	<code>rd.aibacksizech4.value</code>	通道 4/通道 D

3. 电源 (Power Supply)

每台雨珠设备都至少配备了 2 个可通过程序调节电压值的程控电源。此外，雨珠 S 和雨珠 X 还配备有额外的恒压电源。每个型号的具体配置如下表所示：

电源类型	引脚标识	雨珠 2/雨珠 3	雨珠 S	雨珠 S 面包板	雨珠 X
0V~+5V 程控电源	V+	✓	✓	✓	✓
-5V~0V 程控电源	V-	✓	✓	✓	✓
±12V 恒压电源	±12V	-	✓	✓	-
+5V 恒压电源	+5.0V / 5V	-	-	✓	✓
+3.3V 恒压电源	+3.3V	-	-	✓	-

操作说明：

- 程控电源的开关及输出的电压值需要在仪器操场 (IP) 中或使用 IP SDK 的 API 接口配置；
- 恒压电源分为以下 2 种情况：
 - 雨珠 S 和雨珠 X 设备上的电源引脚可直接输出标注的电压值；
 - 雨珠 S 面包板上的电源引脚在打开对应物理开关后，可输出标注的电压值。

```
AnalogIOChannelEnableSet(ch: int = 0, enable: bool = True) -> int
```

打开或关闭程控电源某个通道的电压输出。

参数：

ch	程控电源通道	引脚标识
0	0V~+5V 正电压输出通道	V+
1	-5V~0V 负电压输出通道	V-

enable: True 为打开电压输出，False 为关闭电压输出。

```
AnalogIOChannelNodeSet(ch: int = 0, value: float = 0) -> int
```

设置程控电源某个通道输出的电压值。

参数：

ch	程控电源通道	引脚标识
0	0V~+5V 正电压输出通道	V+
1	-5V~0V 负电压输出通道	V-

value: 电压值（以 V 为单位），取值范围在 [0, 5] 或 [-5, 0] 内。电压值理论上可以是范围内的任意的浮点数值，但在实际使用中会受到设备精度的限制，雨珠设备的程控电压源精度通常在 ±10mV 以内。

4. 任意波形发生器 (Arbitrary Waveform Generator, AWG)

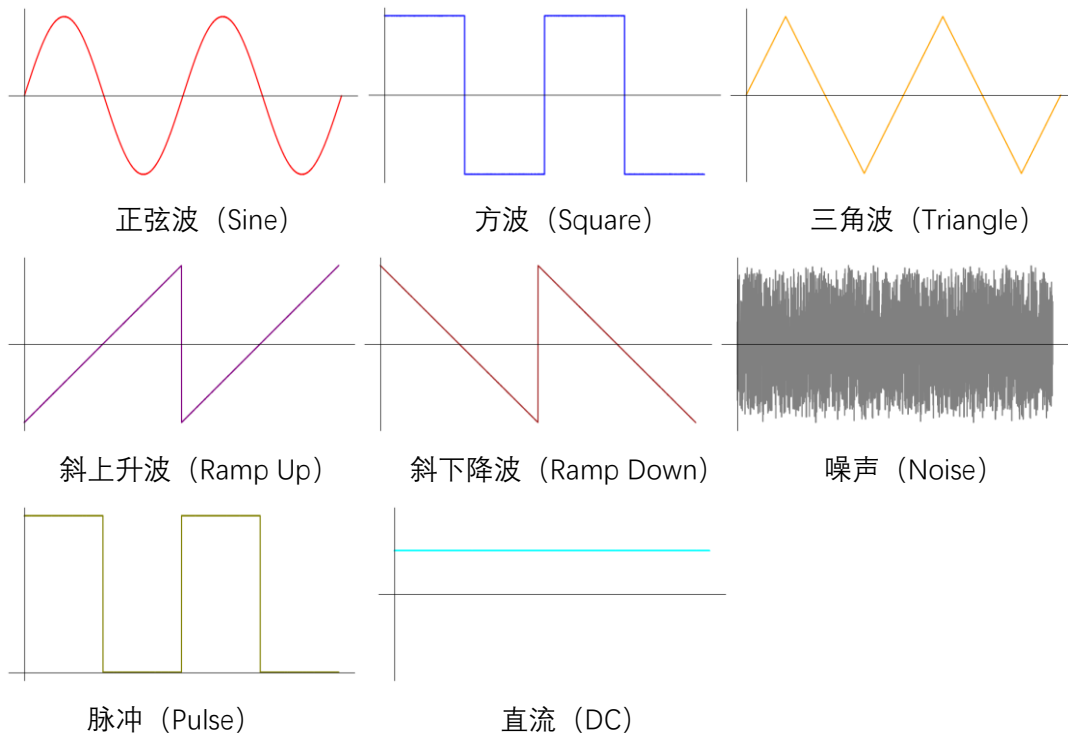
4.1 节点 (Node)

节点是信号生成过程中的可编程单元，允许用户独立控制信号的各个部分，比如载波波形和调制信号。目前，雨霖设备只有载波节点，用于定义未调制的基础波形（如波形类型、频率、幅值等）。未来可能会实装 AM/FM 调制节点，以生成调幅/调频信号。

对于目前所有的 AWG 函数，如果其有 node 参数，则该参数只有以下一种选择：

Node	节点
<code>RDAnalogOutNodeCarrier</code>	载波信号 (Carrier signal)

雨霖设备内置的简单波形函数：



```
AnalogOutNodeEnableSet(ch: int = 0, node = RDAnalogOutNodeCarrier,  
enable: bool = True) -> int
```

启用或关闭信号源某个通道的功能。

参数：

ch	信号源通道	引脚标识
0	通道 1	W1 / AWG1
1	通道 2	W2 / AWG2

node: 使用默认值，详情说明请参考 4.1 节点。

enable: True 为启用，False 为关闭。

```
AnalogOutNodeFunctionSet(ch: int = 0, node = RDAudioOutNodeCarrier,
func = RDFUNCDC) -> int
```

设置信号源对应通道和节点的波形函数，波形函数分为简单波形函数、自定义波形。

参数：

ch	信号源通道	引脚标识
0	通道 1	W1 / AWG1
1	通道 2	W2 / AWG2

node: 使用默认值，详情说明请参考 4.1 节点。

func	波形函数
RDFUNCDC	直流 (DC)
RDFUNCSine	正弦波 (Sine)
RDFUNCsquare	方波 (Square)
RDFUNCTriangle	三角波 (Triangle)
RDFUNCrampUp	斜上升波 (Ramp-Up)
RDFUNCrampDown	斜下降波 (Ramp-Down)
RDFUNCNoise	噪声 (Noise)
RDFUNCPulse	脉冲 (Pulse)
RDFUNCCustom	自定义波形 (Custom)
RDFUNCPlay	播放模式 (Play mode)

run: True 为开始运行，False 为停止运行。

```
AnalogOutNodeFrequencySet(ch: int = 0, node = RDAudioOutNodeCarrier,
hzFrequency: float = 1000.0) -> int
```

设置信号源输出信号的频率，对于不同波形函数类型，频率定义如下：

- 简单波形函数：以一个完整的波形为一个周期；
- 自定义波形函数：以 256 个采样点为一个周期。

参数：

ch	信号源通道	引脚标识
0	通道 1	W1 / AWG1
1	通道 2	W2 / AWG2

node: 使用默认值，详情说明请参考 4.1 节点。

hzFrequency: 频率，单位为 Hz，范围为 0.01Hz – 150kHz。

```
AnalogOutNodeOffsetAmpSet(ch: int = 0, node = RDAAnalogOutNodeCarrier,
vOffset: float = 0.0, amp: float = 1.0) -> int
```

设置信号源输出信号的幅值和偏置。

参数：

ch	信号源通道	引脚标识
0	通道 1	W1 / AWG1
1	通道 2	W2 / AWG2

node: 使用默认值，详情说明请参考 4.1 节点。

vOffset: 偏置，单位为 V，范围为 -5V ~ +5V。

amp: 幅值，单位为 V，范围为 -5V ~ +5V。

```
AnalogOutNodeSymmetrySet(ch: int = 0, node = RDAAnalogOutNodeCarrier,
percentageSymmetry: float = 50.0) -> int
```

未实装的函数。

```
AnalogOutNodePhaseSet(ch: int = 0, node = RDAAnalogOutNodeCarrier,
degreePhase: float = 0.0) -> int
```

设置信号源某个通道的初始相位。

参数：

ch: 通道索引，详细说明参见 AnalogOutNodeEnableSet；

node: 节点索引，详情说明参见 AnalogOutNodeEnableSet；

degreePhase: 初始相位值，以角度（degree）为单位，

```
AnalogOutConfigure(ch: int = 0, output: bool = True) -> int
```

命令信号源某个通道开始或终止输出信号。同步模式下，通道 1 和通道 2 同时输出信号，并且 2 个通道间有固定的相位差；默认情况下，每个通道的初始相位为 0，通道间的相位差也为 0，可以通过函数 AnalogOutNodePhaseSet 调整每个通道的初始相位来改变相位差。

参数：

ch	信号源通道	引脚标识
0	通道 1	W1 / AWG1
1	通道 2	W1 / AWG1
2	同步模式	-

output: 如果为 True，调用函数后相应的信号源通道会开始输出信号；反之（为 False）会终止输出信号。

```
AnalogOutNodeDataSet(ch: int = 0, node = RDAAnalogOutNodeCarrier,
data: list[float] = []) -> int
```

设置自定义波形的数据。自定义波形以 256 个采样点为一个周期，如果给出的数据不足 256 个点，会自动在末尾补 0；给出数据的应归一化，最终输出的信号会乘以幅值。

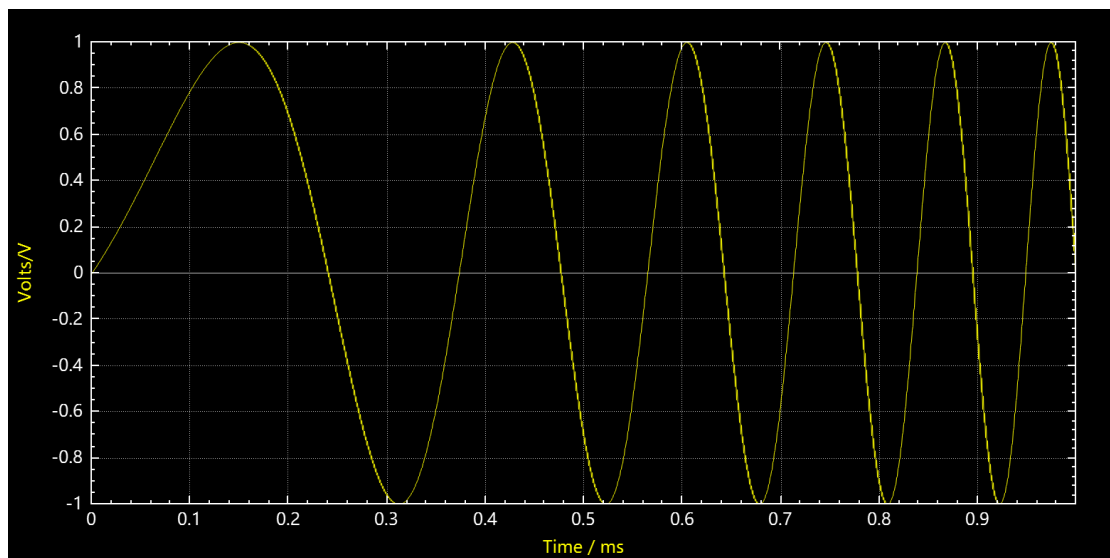
参数：

ch	信号源通道	引脚标识
0	通道 1	W1 / AWG1
1	通道 2	W1 / AWG1

data: 自定义波形数据，长度最大为 256，每个元素的取值范围为[-1, 1]。

4.2 扫描模式 (Sweep mode)

扫描模式指在一个扫描周期内，信号的频率从起始值逐渐变为终止值。扫描模式仅对内置的简单波形有效。比如，如果设定扫描周期为 1ms，扫描的起始频率和终止频率分别为 1kHz 和 10kHz，波形函数为正弦波，则信号源（的某个通道）会以 1ms 为周期输出如下图所示的信号。



使用以下函数启用或关闭扫描模式，以及设置扫描模式下的参数：

```
AnalogOutNodeSweepFreEnableSet(ch: int = 0, enable: bool = True) ->
int
```

启用或关闭信号源某个通道的扫描功能。

参数：

ch	信号源通道	引脚标识
0	通道 1	W1 / AWG1
1	通道 2	W1 / AWG1
2	同步模式	-

enable: True 为启用，False 为关闭。

```
AnalogOutNodeSweepFreSet(ch: int = 0, fre_start: int = 1, fre_stop: int = 100, timebase: float = 1.0) -> int
```

设置扫描模式下信号源某个通道的起始频率，终止频率和扫描周期。

参数：

ch	信号源通道	引脚标识
0	通道 1	W1 / AWG1
1	通道 2	W1 / AWG1
2	同步模式	-

fre_start: 扫描的起始频率，范围为 0~20MHz；

fre_stop: 扫描的终止频率，范围为 0~20MHz；

timebase: 扫描周期，单位为秒（s）。

扫描模式下还可以使用以下函数：

AnalogOutNodeFunctionSet	设置波形函数
AnalogOutNodeOffsetAmpSet	设置幅值
AnalogOutNodePhaseSet	设置初始相位
AnalogOutNodeSymmetrySet	设置对称性（未实装）
AnalogOutConfigure	输出信号

5. 数字静态 IO (Static IO)

5.1 如何访问数字 IO 端口

通常情况下，雨霖设备有 16 个数字 IO 端口 (IO 0 ~ IO 15)，对应设备上标注有 0~15 的 16 个引脚。使用 API 接口访问某个或多个端口时，可通过一个 16 位 (2 字节) 的位掩码 (bit mask) 来控制。位掩码的每 1 位对应 1 个端口，编码方式为 LSB (Least Significant Bit, 最低有效位)，即最低位 (bit 0) 对应 IO 0，最高位 (bit 15) 对应 IO 15，如下表所示：

位 (bit)	15	14	...	1	0
对应 IO	IO 15	IO 14	...	IO 1	IO 0

位掩码的某一位或某几位为 1，表示对应端口被选中，示例如下：

- 0b0000_0000_0000_0001 或 0x0001 表示仅 IO 0 被选中；
- 0b0010_0000_1000_0001 或 0x2081 表示 IO 0，IO 7 和 IO 13 被选中。

使用 Python API 时，位掩码可以是 2 进制、10 进制、16 进制或其他进制。另外，使用 Python 的位操作可以方便地计算位掩码，比如：

- (1 << 3) 的结果是 0b1000，表示 IO 3 被选中；
- (0b0001 | 0b0010) 的结果是 0b0011，表示 IO 0 和 IO 1 被选中。

```
DigitalIOOutputEnableSet(channel: int = 0x00) -> int
```

设置数字 IO 端口是否为输出模式。数字 IO 端口在输出模式下可以持续输出逻辑电平 (高/低)，而在非输出模式下可以作为输入端口，检测输入信号的电平状态。

参数：

channel: 数字 IO 端口的 16 位掩码，取值范围为 0x0000~0xFFFF。当掩码值的某位为 1 时，表示对应的 IO 端口被设置为输出模式。

```
DigitalIOOutputSet(outdata: int = 0x00) -> int
```

设置数字 IO 端口输出的电平状态。

参数：

outdata: 数字 IO 端口的位掩码，取值范围为 0x0000~0xFFFF。当端口被配置为输出模式时，若对应的掩码值为 1，则输出高电平/逻辑 1；若对应的掩码值为 0，则输出低电平/逻辑 0。

```
DigitalIOInputStatus() -> int
```

读取数字 IO 端口的电平状态，结果以 16 位掩码 (int 类型) 保存于 `rd.stiodata.value` 中。若掩码值为 1，表示对应的数字 IO 端口处于高电平/逻辑 1 状态；若掩码值为 0，则表示其处于低电平/逻辑 0 状态。

6. 逻辑分析仪 (Logic Analyzer)

逻辑分析仪是专门用于验证和调试数字电路的工具，与示波器在功能上相似，但也有以下显著差异：

- 输入通道数量：雨珠设备的示波器通常配备 2 至 4 个通道，而逻辑分析仪的输入通道有 16 个；
- 信号采集方式：示波器通过 ADC 对信号进行采样，从而精确还原信号波形和细节，而逻辑分析仪会将信号显示为 2 进制值，其判定依据是测量电压是否高于或低于阈值电压。

DigitalInChannelSet(channel: int = 0x00) -> int

启用或关闭数字 IO 端口的数字输入（即逻辑分析仪）功能。

参数：

channel: 数字 IO 端口的 16 位掩码，取值范围为 0x0000~0xFFFF，取值定义如下：

位掩码中某位取值	对应的数字 IO 端口
0	关闭数字输入
1	启用数字输入

DigitalInDividerSet(div: int = 1) -> int

设置逻辑分析仪的采样率（sample rate），详细说明请参考 2.3 采样率与缓冲区大小。

参数：

div: 分频参数。采样率 = 采样率上限/分频参数，其中采样率上限为 40MHz 或 125MHz，取决于硬件型号（详见 2.3）；分频参数为不大于采样率上限的正整数。

DigitalInBufferSizeSet(bufferize: int = 2048) -> int

设置逻辑分析仪的缓冲区大小（buffer size），详细说明请参考 2.3 采样率与缓冲区大小。

参数：

bufferize: 可选值为 32, 64, 128, 256, 512, 1024, 2048，单位为采样点。

DigitalInTriggerSourceSet(trigsrc = RDTRIGSRCDetectorDigitalIn) -> int

设置逻辑分析仪的触发源。

参数：

trigsrc	触发源
RDTRIGSRCDetectorDigitalIn	数字输入信号（自身）触发
RDTRIGSRCExternal1	外部触发源 1
RDTRIGSRCExternal2	外部触发源 2

```
DigitalInTriggerTypeSet(trigtype = RDTRIGTYPEEdge) -> int
```

设置逻辑分析仪的触发方式。

参数：

trigtype	触发方式
RDTRIGTYPEEdge	边沿触发

```
DigitalInTriggerSlopeSet(trigslope = RDTriggerSlopeEdge) -> int
```

设置逻辑分析仪的触发方向，仅适用于触发源不为 RDTRIGSRCDetectorDigitalIn 的情况。

参数：

slope	触发方向
RDTriggerSlopeRise	上升沿
RDTriggerSlopeFall	下降沿
RDTriggerSlopeEdge	双边沿

```
DigitalInTriggerSet(Rais: int = 0x00, Fall: int = 0x00) -> int
```

当逻辑分析仪的触发源为 RDTRIGSRCDetectorDigitalIn 时，设置各个通道触发边沿开关。

参数：

Rais: 上升沿的开关。

Fall: 下降沿的开关。

```
DigitalInTriggerTimeoutSet(times: int = 0) -> int
```

设置逻辑分析仪的触发超时时间。当超过指定时间时，系统将自动触发扫描。

参数：

times: 触发超时时间，单位为秒（s）；若该值为 0，则永不超时。

```
DigitalInConfigure(enable: bool = True) -> int
```

使逻辑分析仪开始或停止采集信号。

参数：

enable: True 为开始采集，False 为停止采集。

DigitalInStatus() -> **int**

读取逻辑分析仪的当前状态。

返回值：

该函数返回一个 **int** 类型的值，表示设备的当前状态，并且与文件 **RDconstant.py** 中定义的常量对应，具体含义如下：

int	RDconstant	示波器状态
0	RDStateReady	准备完成，等待操作
1	RDStateArmed	等待触发条件满足
2	RDStateDone	信号采集完成
3	RDStateRunning	信号采集中
3	RDStateTriggered	触发条件满足，触发完成
4	RDStateConfig	设备配置中
5	RDStatePrefill	预填充触发前所需的样本缓冲
7	RDStateWait	

DigitalInRead(size: int = 2048) -> **int**

读取逻辑分析仪采集的信号数据，并将结果存储于变量 **rd.didata** 中；读取到的信号数据的长度（即 **rd.didata** 的长度）存储于变量 **rd.dibacksize** 中。

关于变量 **rd.didata** 的说明：

- 数据类型为列表，每个元素表示一个数据帧；
- 元素的数据类型为 16 位无符号整数；
- 元素的值为位掩码，每个比特位对应一个逻辑分析仪通道的状态（0/1）。

参数：

size: 设置用于保存读取到的数据的数组大小，通常应与缓冲区大小相等。如果不相等，则可能会截断读取到的数据或在其末端补 0。

7. 码型发生器 (Pattern Generator)

`DigitalOutEnableSet(chs: int = 0x00) -> int`

启用或关闭数字 IO 端口的数字输出（即码型发生器）功能。

参数：

chs: 数字 IO 端口的位掩码，取值范围为 0x0000~0xFFFF，某位取值定义如下：

位掩码中某位取值	对应的数字 IO 端口
0	关闭
1	启用

`DigitalOutTypeSet(ch: int = 0, dtype = RDDigitalOutTypePulse) -> int`

设置数字 IO 端口输出的码型类型，目前支持的类型有时钟（占空比为 50%的脉冲）、脉冲和随机类型。

参数：

ch: 数字 IO 端口的编号，取值范围为[0, 15]，对应端口 IO 0 ~ IO 15;

dtype:

dtype	码型类型
RDDigitalOutTypePulse	时钟/脉冲
RDDigitalOutTypeRandom	随机

`DigitalOutIdleSet(ch: int = 0, doidle = RDDigitalOutIdleLow) -> int`

设置码型发生器的空闲状态。

参数：

ch: 数字 IO 端口的编号，取值范围为[0, 15]，对应端口 IO 0 ~ IO 15;

doidle:

doidle	空闲状态
RDDigitalOutIdleLow	低电平/0
RDDigitalOutIdleHigh	高电平/1
RDDigitalOutIdleZet	高阻态/Z

`DigitalOutDividerInitSet(ch: int = 0, divinit: int = 0) -> int`

设置码型发生器的初始分频数，用于延迟信号输出。其中分频数定义为

$$\text{分频数} = \frac{\text{可输出的最大信号频率}}{\text{实际输出的信号频率}}$$

雨珠设备可输出的最大信号频率通常为 20MHz。

参数：

ch: 数字 IO 端口的编号，取值范围为[0, 15]，对应端口 IO 0 ~ IO 15;

divinit: 初始分频数。

```
DigitalOutDividerSet(ch: int = 0, div: int = 1) -> int
```

设置码型发生器的初始分频数。其中分频数定义为

$$\text{分频数} = \frac{\text{可输出的最大信号频率}}{\text{实际输出的信号频率}}$$

雨霖设备可输出的最大信号频率通常为 20MHz。

参数：

ch: 数字 IO 端口的编号，取值范围为[0, 15]，对应端口 IO 0 ~ IO 15;

divinit: 分频数。

```
DigitalOutCounterInitSet(ch: int = 0, initlevel: int = 0, counter: int = 0) -> int
```

设置码型发生器某个通道的初始电平和持续时间。

参数：

ch: 数字 IO 端口的编号，取值范围为[0, 15]，对应端口 IO 0 ~ IO 15;

initlevel: 初始电平，0 为低电平，1 为高电平;

counter: 持续时间，单位为输出信号的最小周期（25ns）。

```
DigitalOutCounterSet(ch: int = 0, counter_l: int = 1, counter_h: int = 1) -> int
```

设置码型发生器在一个周期内高低电平的持续时间。

参数：

ch: 数字 IO 端口的编号，取值范围为[0, 15]，对应端口 IO 0 ~ IO 15。

counter_l: 低电平的持续时间，单位为输出信号的最小周期（25ns）。

counter_h: 高电平的持续时间，单位为输出信号的最小周期（25ns）。

```
DigitalOutRun(enable: bool = True) -> int
```

启用或关闭码型发生器的输出功能。

参数：

enable: True 为启用输出，False 为关闭输出。

8. 协议分析仪 (Protocol Analyzer)

1. UART (Universal Asynchronous Receiver/Transmitter)

```
DigitalUartRateSet(rate: float = 9600) -> int
```

设置 UART 串口通信的波特率（即数据传输速率），默认为 9.6kbaud；UART 通信中，接收端和发送端的波特率必须严格一致。

参数：

rate: 要设置的波特率，可从以下数值中选择：

rate		
110	9,600	115,200
150	14,400	128,000
300	19,200	153,600
600	28,800	230,400
1,200	38,400	256,000
2,400	56,000	460,800
4,800	57,600	921,600

```
DigitalUartDIOSet(TxDIO: int = 0, RxDIO: int = 1) -> int
```

将指定的数字 IO 端口配置为在 UART 串口通信中用于发送数据的 Tx 或用于接收数据的 Rx，默认的 Tx 为 IO 0，Rx 为 IO 1。

参数：

TxDIO: 要配置为 Tx 的 IO 端口，可从[0, 15]中选择；

RxDIO: 要配置为 Rx 的 IO 端口，可从[0, 15]中选择。

```
DigitalUartTx(data: list[int] = []) -> int
```

通过 Tx 发送指定的数据。

参数：

data: 待发送的数据，

- 该数据是一个列表，其中每个元素代表一个 8 位的数据帧。在 Python 的 API 中，每个元素用一个取值范围为[0, 255]的整数来表示；
- 单次可发送的最大数据长度为 255 字节，即列表的长度最大为 255；
- 可自行选择数据的编码方式，但接收端必须采用与发送端相同的编解码规则来解码数据。当传输英文字母或数字等文本类数据时，推荐采用 ASCII 编码。例如，字符串"Hello"可表示为：

```
data = [72, 101, 108, 108, 111]
```

其中每个整数对应一个 ASCII 字符，如 72→'H'，101→'e'，...

```
DigitalUartRx(size: int = 255) -> int
```

读取通过 Rx 接收到的数据。

参数：

size: 待接收的数据长度，单位为字节（8 位）；默认值为 255 字节，同时这也是单次可接收的最大数据长度。

2. I²C (Inter-Integrated Circuit)

```
DigitalI2CRateSet(rate: float = 10000) -> int
```

设置 I²C 串口通信的通信速率（bits/s）；通信速率等于时钟线（SCL）上的同步时钟信号的频率（Hz）。

参数：

rate: 要设置的通信速率，可从以下数值中选择：

rate		
1	400	1e5
4	1e3	4e5
10	4e3	1e6
40	1e4	4e6
100	4e4	1e7

```
DigitalI2CADIOSet(SCLDIO: int = 0, SDADIO: int = 1) -> int
```

将指定的数字 IO 端口配置为 I²C 串口通信中的时钟线（SCL）和数据线（SDA）。默认的 SCL 为 IO 0，SDA 为 IO 1。

参数：

SCLDIO: 要配置为 SCL 的 IO 端口，可从[0, 15]中选择；

SDADIO: 要配置为 SDA 的 IO 端口，可从[0, 15]中选择。

```
DigitalI2CTx(data: list[int] = [], addr: int = 0x00) -> int
```

通过 SDA 发送指定的数据。

参数：

data: 待发送的数据，详细说明参考 `DigitalUartTx`；

addr: 待通信的从设备（Slave Device）的地址。

```
DigitalI2CRx(size: int = 255, addr: int = 0x00) -> int
```

读取通过 SDA 接收到的数据。

参数：

size: 待接收的数据长度，单位为字节（8 位）；默认值为 255 字节，同时这也是单次可接收的最大数据长度。

addr: 待通信的从设备（Slave Device）的地址。

3. SPI (Serial Peripheral Interface)

DigitalSPISlaveSet(slave: int = 0) -> int

设置 SPI 通信的主从模式。

参数：

slave	模式
0	从机模式
1	主机模式

DigitalSPIModeSet(mode: int = 0) -> int

设置设备的 SPI 通信模式。

参数：

mode	时钟极性 (CPOL)	时钟相位 (CPHA)
0	0	0
1	0	1
2	1	0
3	1	1

CPOL (Clock Polarity, 时钟极性)

- CPOL=0: SCLK 空闲时为低电平；
- CPOL=1: SCLK 空闲时为高电平；

CPHA (Clock Phase, 时钟相位)

- CPHA=0: 数据在 第一个时钟边沿 (SCLK 的第一个跳变沿) 采样；
- CPHA=1: 数据在 第二个时钟边沿 (SCLK 的第二个跳变沿) 采样。

DigitalSPIRateSet(self,rate: float = 10000) -> int

设置 SPI 通信的通信速率 (bits/s)；通信速率等于时钟线 (SCLK) 上的同步时钟信号的频率 (Hz)。

参数：

rate: 要设置的通信速率，可从以下数值中选择：

rate	rate	rate
1	400	1e5
4	1e3	4e5
10	4e3	1e6
40	1e4	4e6
100	4e4	1e7


```
DigitalSPIDIOSet(CSDIO: int = 0, CLKDIO: int = 1, DQ0: int = 2, DQ1: int = 3) -> int
```

要配置为 SPI 通信中的片选线 (CS)，时钟线 (SCLK) 和数据线 (DQ0, DQ1) 的数字 IO 端口。默认的 SCL 为 IO 0，SDA 为 IO 1。

参数：

CSDIO: 要配置为 CS 的 IO 端口，可从 [0, 15] 中选择；

CLKDIO: 要配置为 SCLK 的 IO 端口，可从 [0, 15] 中选择；

DQ0: 要配置为 DQ0（主设备输出）的 IO 端口，可从 [0, 15] 中选择；

DQ1: 要配置为 DQ1（主设备输入）的 IO 端口，可从 [0, 15] 中选择。

```
DigitalSPIDelaySet(delayCLK: int = 0) -> int
```

设置 SPI 通信的读取延时。

参数：

delayCLK: 读取延时，可从 0, 1, 2, 5, 10, 20 中选择。

```
DigitalSPITx(data: list[int] = []) -> int
```

通过 DQ0 或 DQ1 发送指定的数据。

参数：

data: 待发送的数据，详细说明参考 [DigitalUartTx](#)。

```
DigitalSPIRx(size: int = 255) -> int
```

读取通过 DQ0 或 DQ1 接收到的数据。

参数：

size: 待接收的数据长度，单位为字节（8 位）；默认值为 255 字节，同时这也是单次可接收的最大数据长度。

9. 数字万用表 (Digital Multimeter, DMM)

数字万用表可以快速、精确地获取电路参数。在雨珠系列中，目前有 2 种型号的设备具有数字万用表功能，分别为雨珠 S 和雨珠 X，这 2 种型号的基础测量功能如下：

型号	测量模式	量程
雨珠 S	直流电压 (DCV)	1V, 10V, 100V, 1000V
	直流电流 (DCA)	40mA, 400mA, 3A
	交流电压 (ACV)	1V, 10V, 100V, 1000V
	交流电流 (ACA)	40mA, 400mA, 3A
	电阻 (R)	2kΩ, 20kΩ, 200kΩ, 2MΩ
	二极管 (Diode)	-
雨珠 X	直流电压 (DCV)	50mV, 500mV, 5V, 50V
	直流电流 (DCA)	500μA, 5mA, 50mA, 500mA, 5A
	交流电压 (ACV)	50mV, 500mV, 5V, 50V
	交流电流 (ACA)	500μA, 5mA, 50mA, 500mA, 5A
	电阻 (R)	50Ω, 500Ω, 5kΩ, 50kΩ, 500kΩ, 5MΩ, 50MΩ
	二极管 (Diode)	-

操作方法：

1. 如果测量直流或交流**电压**，将**红色测试线**插入 **V/Ω** 接口；如果测量直流或交流**电流**，按待测电流的预估范围，将**红色测试线**插入 **mA** 或 **A** 接口；如果测量**电阻**，将**红色测试线**插入 **V/Ω** 接口；
2. 将**黑色测试线**插入 **COM** 公共端接口；
3. 选择最大量程，读取初始读数后，再选择合适的量程以提高测量分辨率和精度。

```
DMMOpen(enable: bool = True) -> int
```

打开或关闭数字万用表。

参数:

enable: True 为打开， False 为关闭。

```
DMMSet(dmmtype = RDDMMDCV, rangeOfIndex: int = 0) -> int
```

设置数字万用表的测量模式和量程。

参数:

dmmttype	测量模式
RDDMMDCV	直流电压 (DCV)
RDDMMDCA	直流电流 (DCA)
RDDMMACV	交流电压 (ACV)
RDDMMACA	交流电流 (ACA)
RDDMMR	电阻 (R)

RDDMMDiode		二极管（Diode）	
rangeOfIndex: 量程的索引值，与型号、测量模式和量程有关，参见下表：			
型号	测量模式	量程	rangeOfIndex
雨珠 S	直流电压（DCV） 交流电压（ACV）	1V	0
		10V	1
		100V	2
		1000V	3
	直流电流（DCA） 交流电流（ACA）	40mA	0
		400mA	1
		3A	2
	电阻（R）	2kΩ	0
		20kΩ	1
		200kΩ	2
		2MΩ	3
	二极管（Diode）	-	0
雨珠 X	直流电压（DCV） 交流电压（ACV）	50mV	0
		500mV	1
		5V	2
		50V	3
	直流电流（DCA） 交流电流（ACA）	500μA	0
		5mA	1
		50mA	2
		500mA	3
		5A	4
	电阻（R）	50Ω	0
		500Ω	1
		5kΩ	2
		50kΩ	3
		500kΩ	4
		5MΩ	5
	50MΩ	6	
	二极管（Diode）	-	0

`RDDMMReadSingle()` -> `int`

读取数字万用表的测量结果。结果以 `byte` 类型保存在 `self.DMMData.value` 中, 该字符串的长度保存在 `self.DMMbacksize.value` 中。例如, 若选择了 5V 量程测量电压, 结果可能是 `b"3.1415V"`。